Distributed Systems: **A Report on CHORD**

FRANCESCO SAVERIO LACCHÈ

July 2026

Contents

1 Introduction

The increasing scale of modern distributed applications has highlighted the limitations of traditional centralized architectures when operating under highly dynamic workloads. Large-scale services require systems capable of efficiently supporting a continuously growing number of users while ensuring scalability, fault tolerance, availability and balanced resource utilization. These requirements have motivated the development of alternative architectural models that distribute both data and computation across multiple participants. Peer-to-Peer (P2P) systems represent one of the most significant examples of this approach, as they eliminate the dependency on centralized components by distributing resources and responsibilities among participating peers.

1.1 Centralized Architectures

A centralized system is defined by a primary node, typically a server or a mainframe, surrounded by secondary nodes, typically clients, that interact with it. The distinguishing factor is that there is asymmetry between these two roles: one that provides the service and one that receives it, one that owns and manages the data and the one that requests it.

Although the logical architecture is centralized, modern client-server systems are often implemented using multiple physical machines organized according to layered or multi-tier architectures. Different servers may be responsible for process logic, caching or data persistence. Consequently, the line that separate a client from a server is often blurred [1].

1.1.1 Client-Server Model

We take into account the most relevalnt centralized model, the client-server. Formally, we let

$$C = \{c_1, c_2, \ldots, c_n\}$$

denote the set of clients and

$$S = \{s_1, s_2, \ldots, s_m\}$$

denote the set of servers. Generally, in a synchronous communication context, clients interact with servers through a request-response communication model

$$c_i \longrightarrow s_j$$

where a client submits a single request to a single server, which execute it maybe interacting with other servers and returns the corresponding response back to the client. In this context, servers are often designed as aggregators (API gateways). For this, the difference between client and server is not always a clear cut.

1.1.2 Advantages and Disadvantages

Reasons to implement a centralized architectures are many [1].

First, maintaining a single logical component managing the data simplifies consistency management. Since the primary copy of each resource is managed by the server infrastructure, distributed consistency protocols are generally unnecessary or considerably simplified by using centralized solutions.

Second, centralized systems avoid many forms of distributed coordination. Operations such as distributed synchronization, Lamport's clocks, distributed mutual exclusion or consensus algorithms become generally unnecessary because computation data are boundend to a single component.

Third, centralized architectures are generally easier to design and maintain. Admin operations such as software updates, monitoring and backup procedures can be performed from a single management point, considerably reducing mantainance effort.

Despite their simplicity, centralized architectures exhibit several limitations that become increasingly significant as the system grows that often makes their use impracticable [1] [2].

In fact, **the most evident limitation concerns scalability**. Since computational resources are concentrated within the server infrastructure, every client request eventually converges toward a single server or limited set of servers. As the number of users grows, system may face performance bottlenecks or crash.

A second limitation is the presence of **potential single points of failure**. Although practical systems often employ replication and redundancy to make the system more tolerant to fault, the logical organization of the service resides typically on single, centralized component whose failure may degrade or interrupt the service.

Although centralized architectures remain suitable for many distributed applications, their limitations become increasingly significant in large-scale and highly dynamic environments. In particular, the need to improve scalability, eliminate centralized bottlenecks and increase fault tolerance motivated the development of alternative distributed architectures.

Among these, Peer-to-Peer (P2P) were the one mostly used.

1.2 P2P Architectures

Peer-to-Peer (P2P) architectures were introduced as an alternative to the centralized systems. They are belong to the family of decentralized systems, meaning that rather than concentrating computation and storage within a central server, they decentralize these responsibilities among all participating components, commonly referred to as *peers* or *nodes* [1].

The main characteristic of a P2P architecture is the symmetry between participants. Each peer can simultaneously act as both a provider and as a consumer of computational power, storage capacity, communication bandwidth or, in general, resources to the overall system. In these architectures, there is an inversion: the more users participate in the network, the higher the performance the system has, which is the opposite direction with respect centralized solutions.

In fact, unlike centralized architectures, where network's adaptations are barely tolerated, P2P systems are designed under the assumption that nodes may join, leave or fail at any time. The overlay must continuously adapt to changes in membership while keeping untouched its operational capabilities. This dynamic behavior, commonly referred to as *churn*, represents one of the principal challenges in the design of large-scale Peer-to-Peer systems. It follows that, the design of P2P systems intrisically gives strong capabilities about fault tolerance.

Depending on the degree of decentralization adopted, P2P systems can be classified into two orthogonal dimensions [1] [2]:

- **Degree of Decentralization**
  Defines whether the system relies exclusively on participating peers or introduces specialized node.
  - ***Pure Peer-to-Peer systems***, where every participating node has identical responsibilities.
  - ***Hybrid Peer-to-Peer systems***, where specialized nodes assist basic peers for some operations.
- **Overlay Organization**
  Defines if peers are arranged according to a deterministic topology or not.
  - ***Structured Peer-to-Peer systems***, where peers are organized according to a deterministic topology, allowing resources to be located through efficient and known routing algorithms.
  - ***Unstructured Peer-to-Peer systems***, where peers establish arbitrary logical connections and resource discovery is generally done through mechanisms like flooding or random walks.

1.2.1 Pure/Hybrid Peer-to-Peer Systems

According to the degree of decentralization adopted, it is possible to classify P2P architectures into two categories [1].

In a **Pure Peer-to-Peer** architecture, all participating nodes have identical responsibilities and capabilities. There are no dedicated component or centralized coordinators and every peer contributes equally to the operation of the distributed system. Every peer performs the same logical functions and has the same rights of any other one. Can be seen as a democratic model. Each peer is responsible to manage resource storage, request processing and message forwarding equally. Consequently, no node constitutes a privileged component whose failure compromises the operation of the entire system. As additional peers join the network, both the available computational resources and the aggregate storage capacity increase proportionally.

A **Hybrid Peer-to-Peer** architecture, instead, preserves the decentralized exchange of resources while introducing a limited number of specialized nodes, commonly referred to as *superpeers*. These nodes extend the functionality of ordinary peers by maintaining additional routing information, indexing available resources and facilitating peer discovery. Their objective is to assist the network's basic peers in their tasks, with the ultimate goal of improving efficiency.

1.2.2 Structured/Unstructured Peer-to-Peer Systems

A second classification of Peer-to-Peer systems concerns the organization of the overlay network [1].

In a **Structured Peer-to-Peer** system, participating peers are organized according to a deterministic overlay topology. Consequently, every node occupies a well-defined logical position within the network, allowing routing decisions to be performed through predefined algorithms. As we will see, these identifiers are managed via specific protocols, of which DHTs are a part. The use of a topology is often motivated by efficiency issues: whenever a user wants to retrieve a certain resource, it faces a well-defined topology. Resource placement is therefore deterministic, meaning that the location of a resource is uniquely determined by its identifier rather than by search procedures. This organization enables efficient and predictable lookup operations while requiring each peer to maintain only a limited amount of routing information.

On the other hand, in an **Unstructured Peer-to-Peer** system, peers establish logical connections without following any predefined topology (random graph). Since resources are not assigned according to deterministic rules, their location cannot be inferred directly from their identifiers. Instead, resource discovery relies on search mechanisms such as flooding, random walks or epidemic spreding.

1.2.3 Requirements for Large-Scale P2P Architectures

The distinctions made before, especially about structured and unstructured architectures, highlighs many differences between the two approaches. In this context, independently on the chosen configuration, the requirements that is wanted to acheive over a P2P architecture can be classified as a set of five points [2]. They are briefly described below.

First, resource discovery must be efficient. Locating a resource by contacting every node is clearly laborious in networks composed of thousands or millions of participants. Reasoning a bit on this property, we can understand that structured architectures can generally be better to unstructured ones for this point. This last statement is based on the observation that searching for a resource within a structured P2P architecture generally occurs in logarithmic time O(log N) while in an unstructured P2P takes linear time O(N).

Second, the system must be scalable. In this context, this means that, as the number of peers increases, the system must not slow down. Therefore, the objective is to make the system resistent to from large-scale use, ensuring that the number of users does not affect performance. Therefore is aimed maintain this lookup complexity ideally bounded to logarithmic times (for structured architectures) or linear (for unstructured ones) as N grows.

Third, the system must be able to proprely manage *dynamic membership* (or *churn*). Since by assumption peers may join or leave the network at any moment, the overlay must continuously adapt while preserving its routing capabilities constant. Of course, considering not structured architecture it is not necessary to address this issue, given that there is no pre-established overlay. This point is, somehow, referred only to structured ones.

Fourth, the system must avoid resource concetration. In this context, this means that resources should be evenly distributed among participants to avoid load concentration on a limited number of nodes. Effective load balancing therefore becomes a key design objective. This is related to the fact that, large-scale P2P systems must also exhibit fault tolerance by continuing to operate correctly despite node failures or network partitions.

Fifth, the systems should remain fully decentralized, eliminating the need for centralized coordinators or the awareness of the global system while allowing every peer to participate equally in the management of the overlay.

All of these requirements motivated the development of *Distributed Hash Tables (DHTs)*, a data structure used on **structured Peer-to-Peer systems** like Chord, designed to provide deterministic, efficient, scalable and decentralized resource lookup.

2 Distributed Hash Tables

Previously were highlighted the limitations of both centralized and not structured P2P architectures. While the first mainly suffers from scalability bottlenecks and single points of failure, the second rely on flooding-based lookup mechanisms whose communication cost linearly with the number of peers. These limitations motivated the development of *Distributed Hash Tables* (DHTs), a distributed data collection specifically designed for structured P2P systems [2].

A DHT is an abstract, decentralized collection provides an indexing mechanism that enables deterministic resource placement and efficient lookup without requiring any node to maintain complete knowledge of the overlay topology.

An abstract *Distributed Hash Table* can be formally modeled as the tuple:

$$\mathcal{D} = (N, I, \rho)$$

- $N = \{n_1, n_2, \ldots, n_N\}$ denotes the finite set of participating peers.
- $I = \{0, \ldots, 2^m - 1\}$ denotes the identifier space. It is generally chosen as a cyclic space of size 2^m, where m denotes the number of bits produced by the choosen hash function.
- $\rho : I \to N$ is the *resource assignment function*, associating each key with the node responsible for storing the corresponding resource.

Unlike conventional distributed databases, a DHT does not require global knowledge of the network topology. Instead, each node stores only partial routing information while collectively maintaining a global distributed indexing structure.

Lookup Function - The main operation supported by every DHT is the lookup function (or *retraival function*), which, given a resource identifier $k \in K$, returns the node responsible for storing that resource. Formally:

$$lookup : I \to N$$

$$\forall k \in I \qquad lookup(k) = \rho(k)$$

Lookup Correctness - A DHT is said to provide correct lookup if every lookup operation eventually terminates and returns the unique correct node responsible for the requested key [2].

A DHT can be therefore seen as distributed data structure that allows to efficiently send lookup requests throughout the network and every peer maintains only a subset of the routing information that enables resources to be located in a bounded number of routing hops. The main objective of a DHT is therefore to distribute data and lookup responsibilities uniformly and efficiently across participating peers of the overlay network.

2.1 Hashing

The deterministic mapping between stored resources and participating peers is obtained through hashing functions, which associates an arbitrary string object with an string identifier belonging to the identifier space [3].

$$H : \{s\}^* \longrightarrow \{0, \ldots, 2^m - 1\}$$

In the general approach, the same hash function is generally applied to create references to both data objects and participating nodes. Therefore we assume

$$ID_{resource} = H(resource)$$

and

$$ID_{node} = H(node\ identifier)$$

where the exact meaning of *node identifier* depends on the specific implementation. Generally, is the concatenation (IP+port) or, in some cases the URL.

Hashing Functions Properties - The effectiveness of a DHT strongly depends on the statistical properties of the adopted hash function. There are many variants of hashing functions, often distinguished by the size of the *digest* (output, or in this context, identifier) produced. However, independently on the variant used, a generic hashing function always guarantee:

- **Determinism** - Same inputs always produce same digest. Guarantees that both resources and participating nodes are always assigned the same identifier, ensuring that every peer computes an identical value.
- **Fixed Output Size** - The function always generates a digest of a constant length, regardless of whether the input is a single letter or an entire book. Permits to defines a finite identifier space, allowing routing algorithms to operate over a well-defined domain.
- **Uniformity** - Digest are approximately uniformly distributed over the identifier space. Guarantees load balancing, since it allows resources to be assigned to peers with approximately equal probability.
- **Pre-image Resistance** - Given a digest, it is computationally infeasible to compute an input whose hash equals to such digest. No collisions.
- **Second Pre-image Resistence** - Given an input, it is computationally infeasible to find a different input x producing the same digest. Mainly leveraged for security purposes.

Taken together, these properties allow the DHT to properly work and assinging identifier among peers and resources.

2.2 Fundamental Operations

From an abstract point of view, a DHT behaves as a simple dictionary, providing a minimal set of dictionary operations. Although different implementations uses different routing algorithms and overlay maintenance protocols, they all expose essentially the same interface [1] [2]. Abstract operations are listed below.

Lookup - The lookup operation constitutes the fundamental operation provided by every DHT. Given a key $k \in I$, it determines the node responsible for storing the corresponding resource.

$$lookup : I \to N$$

such that

$$\forall k \in I \qquad lookup(k) = \rho(k)$$

where ρ denotes the resource assignment function introduced previously.

Create (post) - The put operation stores a resource associated with a given key. Let V denote the space of all possible values.

$$post : I \times V \to \emptyset$$

Conceptually, the protocol first executes the lookup operation to determine the responsible node and then stores the key-value pair on that node.

Retrieve (get) – The get operation get the value linked to a given key. Internally, this operation first locates the responsible node through the lookup function before returning the associated value.

$$get : I \to V$$

Delete - The delete operation removes the resource associated with a given key from the distributed hash table.

$$delete : I \to \emptyset$$

NOTE that, as anticipated, the operations described above define the abstract interface exposed by every DHT. In fact, every implementation uses protocol-specific operations required to maintain the specific overlay network that it uses. For example, Chord extends this interface by adding operations as *join()*, *notify()* and *fix_fingers()* whose objective is not to manage stored data directly but to preserve the correctness of the overlay despite dynamic changes.

2.3 Overlay Networks

Since now were used the term *overlay network*, or simply *overlay*. This terms refer to the logical communication structure where the system is deployed and then where the DHT is employed. Unlike the underlying physical network, which is determined by internet connectivity, the overlay defines only the logical neighborhood relationships used to perform routing operations [1] [4].

Formally, the overlay can be represented as the directed graph

$$G = (N, E)$$

where

$$E \subseteq N \times N$$

is the set of edges (logical links) maintained by the routing protocol.

The existence of an overlay network allows routing decisions to be completely independent of the underlying physical topology. Two nodes that are neighbors in the overlay are not necessarily geographically close nor directly connected in the physical network. On the other hand, physically adjacent machines may be located in distant regions of the logical identifier space.

In each structured system, each node maintains routing information concerning only a limited subset of the participating peers. Let

$$deg(v)$$

denote the number of routing entries maintained by node v. An efficient Distributed Hash Table should satisfy

$$deg(v) \ll |N|$$

thus avoiding the need for global knowledge of the overlay.

The organization of the overlay graph distinguishes the various DHT implementations proposed in the literature. Different protocols adopt different routing geometries, including ring-based overlays, multidimensional Cartesian spaces, prefix-based routing, and XOR metrics. Specific DHTs implementations are then choosen consequently.

3 CHORD

Chord is one of the mostly know pure and structured Peer-to-Peer protocols. Implements an abstract DHT by organizing participating peers into a logical ring and providing efficient logarithmic-time resource lookup through consistent hashing and distributed finger (routing) tables [1] [2] [4] [5].

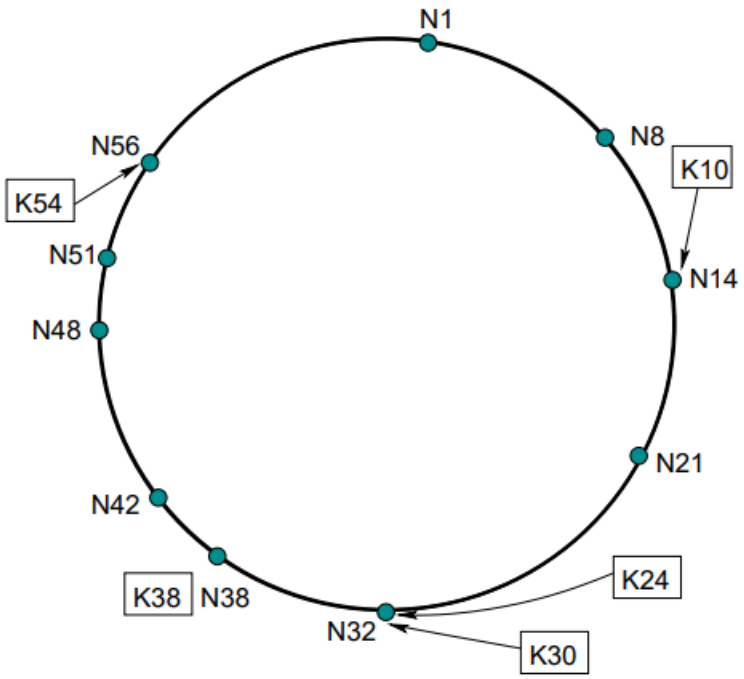

Figure 1: Chord ring overlay [2]

The definition of Chord is based on three fundamental milestones:

1. **Each identifiers are logically organized into a cyclic space overlay, employing modulo $|2^m|$, referred to as the *chord ring***. As a consequence, the overlay assumes a this circular shape.

2. **Each partecipating node is assigned a unique identifier belonging to a finite identifier space**. Resources are deterministically assigned and managed to peers through a consistent hashing mechanism. To know how such indetifiers are created check the section 2.

3. **Each node maintains only partial routing knowledge of the overlay** while collectively providing complete routing functionality.

It now given an formalization of this protocol. As before, we formally let

$$N = \{n_1, n_2, \ldots, n_{|N|}\}$$

denote the set of participating peers. The overlay maintained by Chord can be modeled as the directed graph

$$G = (N, E)$$

where

$$E \subseteq N \times N$$

represents the logical routing links maintained by the protocol.

The identifier space is defined as the cyclic interval

$$I = \{0, 1, \ldots, 2^m - 1\}$$

where m denotes the length of the digest produced by the adopted hash function.

The cyclic nature of the identifier space implies that arithmetic operations are performed modulo $|2^m|$, causing the maximum identifier to wrap around to zero. As a consequence, every node and every resource occupies a unique position on a logical ring rather than on a linear address space. An important design objective of Chord, acheived through DHT, is to minimize the routing state maintained by peers. Maintaining complete knowledge of all participating nodes would require each peer to store $O(N)$ routing entries, making the system impractical at large-scale. Instead, Chord split routing responsibilities among participants, making each peer maintaining averagely O(log N) routing entries [2].

3.1 Consistent Hashing

The fundamental design choices of Chord is the adoption of *consistent hashing*, a distributed hashing technique that provides deterministic resource placement while efficiently supporting dynamic changes of the overlay [1] [2] [5].

The problem addressed by consistent hashing arises from the highly dynamic nature of P2P systems. In such environments, it is assumed that nodes may continuously join or leave the overlay, causing the set of participants to change over time. Traditional hashing techniques are not designed for this scenario: since the destination of a resource depends on the total number of storage locations, adding or removing a node generally cascade the changes of the references of almost every resource, requiring a complete redistribution of the stored data.

Consistent hashing overcomes this problem by making resource assignment largely independent of the current number of participating nodes. Instead of recomputing the placement of every resource after each topology change, only the resources belonging to a limited identifier interval are relocated, while all the remaining remains unchanged. As introduced in Section 2, both participating nodes and stored resources are mapped into the same identifier space by applying the same hash function.

The resulting identifiers are interpreted as positions on the ring. Since the identifier space is cyclic, arithmetic is performed modulo 2^m, causing the maximum identifier to wrap around to zero. Once identifiers have been computed, Chord assigns each resource to the first node encountered while traversing the ring in the clockwise direction. Formally, the node responsible for a resource identified by k is defined by the successor function

$$\rho(k) = successor(k)$$

where $successor(k)$ denotes the first node whose identifier is greater than or equal to k. If no such node exists, the search wraps around the identifier space and continues from identifier 0.

Example - considering the ring shown in Figure 1, a resource whose identifier is 24 is assigned to node 32, while a resource identified by 54 is assigned to node 56. Likewise, a resource with identifier 60 is assigned to node 1, remarking the cyclic characteristic of the circular identifier space. Suppose now that node 32 unexpectedly leaves the overlay. Since node 32 was responsible for all keys belonging to the interval 21 - 32, only ranged resources are affected. These keys are immediately reassigned to the next node on the ring, namely node 38. Every other resource in the system preserves its original assignment, since the successor of their identifiers remains unchanged. Conversely, when a new node joins the overlay, the process occurs in the opposite direction. Assume, for example, that a new peer with identifier 26 enters the overlay, between nodes 21 and 32. Before the insertion, node 32 was responsible for every key in the interval 21 - 32. After the join operation, the new node becomes responsible only for the subinterval 21 - 26 while node 32 is resposible for the subinterval 26 - 32. All remaining resources preserve their previous assignment.

This locality property represents the principal advantage of consistent hashing. Assuming a uniform distribution of nodes and resources, the expected number of relocated keys after a join or leave operation is

$$O\left(\frac{K}{N}\right)$$

where K denotes the number of stored resources and N the number of participating nodes.

3.2 Identifier Space

Once node and resource identifiers have been generated through the hash function, Chord interprets them as positions on a cyclic identifier space [1] [2]. Unlike a conventional linear address space, the identifier space here is cyclic, meaning that the largest identifier is immediately followed by the smallest one. The identifier space is defined as

$$I = Z_{2^m} = \{0, 1, \ldots, 2^m - 1\}$$

where m denotes the number of bits produced by the adopted hash function and Z_{2^m} denotes the cyclic ring of integers modulo 2^m. Consequently, every arithmetic operation performed on identifiers is implicitly computed modulo 2^m. That is, $\forall a, b \in I$,

$$(a + b) \bmod 2^m \in I$$

Consequently, identifier $2^m - 1$ is followed by identifier 0, forming a logical ring. The circular organization induces a total clockwise ordering among all participating peers. Every node has exactly one immediate successor and one immediate predecessor on the ring. The **successor** of a node n, denoted by

$$successor(n)$$

is defined as the first participating node encountered while traversing the identifier space in the clockwise direction. Formally,

$$successor(n) = \min \{x \in N \mid x > n\}$$

where the search wraps around to identifier 0 whenever no node with a larger identifier exists. Similarly, the **predecessor** of node n, denoted by

$$predecessor(n)$$

is the first node encountered while traversing the identifier space in the counterclockwise direction. The successor and predecessor relationships partition the identifier space into a collection of pairwise disjoint intervals. Specifically, every node n is responsible for all identifiers belonging to the interval

$$(predecessor(n),\ n]$$

This interval is called the *responsibility interval* of node n. Consequently, every resource whose identifier belongs to this interval is stored at node n.

Example - consider again the Figure 1. Node 32 has predecessor 21 and successor 38. Therefore, node 32 is responsible for every resource whose identifier belongs to the interval 21 (excluded) - 32 (included). In the same way, node 1 stores every resource whose identifier belongs to 56 (excluded) - 1 (included). This is the wrap-around property.

The successor relationship allows the *resource assignment function* introduced in section 2 to be denoted as

$$\rho(k) = successor(k)$$

meaning that every resource is assigned to the first node encountered in the clockwise direction whose identifier is greater than or equal to the resource identifier.

The correctness of this mechanism therefore relies on a small number of fundamental properties:

- **Every participating node has exactly one successor**.
- **Every participating node has exactly one predecessor**.
- **Every identifier in the identifier space belongs to the responsibility interval of exactly one node**.
- **The collection of responsibility intervals forms a complete partition of the identifier space**.

As long as these properties are preserved, every resource has a unique owner and every lookup operation eventually reaches the node responsible for the requested identifier. The remaining components of the Chord protocol, including finger tables and stabilization procedures, are specifically designed to guarantees these properties efficiently despite nodes continuously joining and leaving the overlay.

3.3 Finger Tables

The successor relationship introduced before is sufficient to guarantee the correctness of resource lookup: a node could lookup any resource by repeatedly forwarding the request to its immediate successor until reaching the node responsible for the requested identifier. Although correct, such an approach is clearly inefficient. In the worst case, a lookup request require

$$O(N)$$

making the protocol not that much efficient for large-scale situations.

To overcome this problem, Chord augments each node with additional routing information called the *finger table* [2]. The purpose of the finger table is to accelerate the lookup process, trying to making lower the linear cost, by allowing requests to "jump" across the identifier space instead of advancing one successor at a time. As before, let

$$I = \{0, 1, \ldots, 2^m - 1\}$$

denote the identifier space. In every peer n, it is stored a local routing table composed of at most m entries. The i-th entry of the finger table is defined as

$$finger[i] = successor\left(n + 2^{\,i-1}\right) \qquad 1 \leq i \leq m$$

where every arithmetic operation is performed modulo 2^m. Intuitively, the i-th finger points to the first node encountered after moving 2^{i-1} positions clockwise along the identifier ring. Consequently, nearby fingers reference nodes that are logically close to the current peer, whereas higher-index entries reference increasingly distant regions of the identifier space.

Each finger table entry therefore stores two pieces of information:

- Starting identifier
$$start_i = (n + 2^{i-1}) \bmod 2^m$$
- Peer responsible for that identifier
$$finger[i] = successor(start_i)$$

NOTE that the start identifier does not necessarily correspond to an existing peer. Instead, Chord always stores the first participating peer whose identifier is equal to or follows the starting position.

Example - Let's again consider Figure 1 and suppose that m = 6 so that the identifier space contains

$$2^6 = 64$$

possible identifiers. Now focus on a certain peer, let's say n = 8. The first finger table entry of it, m = 0, is computed as

$$start_1 = (8 + 2^0) \bmod 64 = 9 \bmod 64 = 9$$

Since the first node encountered clockwise after identifier 9 is node 14.

$$finger[1] = 14$$

Following the same logic, the second entry, m = 1, is computed as

$$start_2 = (8 + 2^1) \bmod 64 = 10$$

whose successor is again node 14. In the same way third entry is computed as

$$start_3 = (8 + 2^2) \bmod 64 = 12$$

which also maps to node 14. Skipping step 4,5 we compute as the m = 5 entry,

$$start_6 = (8 + 2^5) \bmod 64 = 40$$

whose successor is node 42. The resulting finger table therefore contains routing information toward progressively more distant regions of the overlay.

NOTE that the exponential spacing of finger table entries constitutes the fundamental idea behind Chord. Rather than storing information about every participating node, each peer stores shortcuts whose distances increase as powers of two. Consequently, during a lookup operation, a request can rapidly approach the destination by repeatedly jumping the finger that most closely precedes the desired identifier.

Example - consider again node 8 that wishes to locate a resource whose identifier is 54. Using only successor pointers, the lookup would follow the path

$$8 \to 14 \to 21 \to 32 \to 38 \to 42 \to 51 \to 56$$

requiring a O(n) cost.

Instead the finger table approach, node 8 immediately selects its largest finger preceding 54, that is node 42. Node 42 then forwards the request to node 51, which finally reaches node 56.

$$8 \to 42 \to 51 \to 56$$

requiring a O(log n) cost.

This logarithmic routing approach represents the core advantages provided by Chord protocol. By maintaining only partial knowledge of the overlay, each node is able to efficiently locate any resource in the distributed system at logarithmical time.

3.4 Lookup Algorithm

The primary operation supported by Chord is the lookup procedure. Given a certain resource identifier

$$k \in I$$

its objective is to determine the unique peer responsible for storing that resource, denoted as

$$\rho(k) = successor(k)$$

The lookup procedure must satisfy two fundamental requirements [1] [2] [4].

- **Correctness** - it must always return the correct successor of the requested identifier.
- **Efficiency** - It should minimize the number of exchanged messages in order to preserve scalability as the network grows.

As discussed previously, the successor relationship alone is sufficient to guarantee correctness. However, relying exclusively on successor pointers implies to relax efficiency and always performing lookup in linear time. As it was said, Chord uses the finger tables to making more efficient such process.

Naive Lookup - Assume that every node maintains only the link of its successor. Whenever a peer receives a lookup request for identifier k, it first checks whether k belongs to the responsibility interval of its successor:

- **If yes**, it means that the correct node is found and the successor is responsible for the requested resource. The process terminates.
- **Otherwise**, the request is forwarded to the successor, which repeats exactly the same procedure.

In other words it means that the lookup follows the sequence

$$n \rightarrow successor(n) \rightarrow successor(successor(n)) \rightarrow \cdots$$

until the destination node is reached.

Although is correct, this algorithm in the worst case it costs lineraly with respect of the number of peers. We can do it better.

Finger-Based Lookup - A way to improve the previous solution is to employs finger tables. Instead of forwarding the request to the immediate successor, a node selects the finger whose identifier most closely precedes the target identifier while remaining smaller than it. Let

$$closest\_preceding\_finger(k)$$

denote the finger entry having the largest identifier satisfying

$$finger[i] \in (n, k)$$

The lookup algorithm proceeds as follows. If the requested identifier belongs to the interval

$$(n, successor(n)]$$

the successor is immediately returned. Otherwise, the request is forwarded to

$$closest\_preceding\_finger(k)$$

which recursively executes the same procedure until the responsible node is found.

This behaviour is equivalently explain the below provided pseudocode.

```
FUNCTION Lookup(key)
    IF current node is responsible for key THEN
        RETURN current node
    END
    IF successor is responsible for key THEN
        RETURN successor
    END
    next     best finger preceding key
    FORWARD lookup request to next
END
```

```
FUNCTION BestFinger(key)
    Search the finger table
    Select the finger closest to key without passing it
    IF no suitable finger exists THEN
        RETURN successor
    END
END
```

3.5 Join/Leave Operations

One of the key design objectives of Chord is maintaining the correctness of the overlay despite nodes continuously joining, leaving or even crashing. These topology changes, known as *churn*, must compromise as less as possible both resource assignment and lookup operations. Thanks to consistent hashing, only a limited portion of the identifier space is affected by each topology change, allowing the overlay to evolve efficiently [1] [2].

Peer Join - Suppose that a new peer with identifier n wants to join the overlay. The joining node must initially know the address of at least one peer already participating in the network, commonly referred to as the *bootstrap node*. Since Chord is a pure structured P2P architecure, such node does not play any privileged role. It simply provides an entry point into the overlay.

The join procedure consists of four main steps.

1) Locating the successor - The joining peer performs a lookup for its own identifier

$$successor(n) = lookup(n)$$

thus finding the already present node that is currently responsible for the identifier interval containing n.

2) Finger Table Construction - Once n's successor has been identified, the joining node n initializes its successor pointer and constructs its finger table using the lookup procedure described in section 3.3.

3) Resource transfer - After locating its successor, the joining node becomes responsible for the identifier interval

$$(predecessor(successor(n)),\ successor(n)]$$

After the insertion, the new node becomes responsible for

$$(predecessor(n),\ n]$$

Accordingly, only the resources whose identifiers belong to this interval are transferred from the successor to the new node. This redistribution is then local, thanks to consistent hashing.

4) Updating routing information - Finally, neighboring nodes progressively update their successor pointers, predecessor pointers and finger tables, through stabilization, in order to incorporate the new participant into the overlay.

Peer (voluntarily) Leave - A node may leave the overlay either voluntarily or because of an unexpected failure. In a voluteer scenario, the leaving node first transfers all resources belonging to its responsibility interval to its immediate successor, merging them. Subsequently, it informs both its predecessor and successor to his exit so that they can reconnect directly.

Formally, if node n leaves the overlay, all identifiers belonging to

$$(predecessor(n),\ n]$$

become resposibility of the interval of

$$successor(n)$$

Consequently, the successor becomes the new owner of all resources previously managed by node n.

Peer (forces) Leave - A more challenging situation occurs when a node fails unexpectedly. Since crash is unexpected and no notification is generated, neighboring peers during execution continue attempting to communicate with the failed node until the failure is detected. Chord uses RPC replies within a timeout interval (heartbeat RPC) to detect failures. During this interval, parts of the routing information become temporarily inconsistent, potentially causing lookup operations to fail or follow longer paths. Whenever a failure occurs, only the routing information referring to the failed node becomes invalid, while the remaining overlay continues operating correctly. Once the failure is detected, neighboring peers update their routing information and reassembles the ring by redirecting successor and predecessor pointers toward the next participant.

Although the join and leave procedures correctly describe how nodes enter and exit the overlay, they assume that routing information is immediately updated throughout the network. In practice, this assumption rarely holds because routing tables are distributed among all participating peers.

For such problem, the system introduces a collection of periodic maintenance procedures, known as the *stabilization protocol*, whose objective is to progressively restore the consistency of successor pointers, predecessor pointers and finger tables after any kind of topology changes.

3.6 Stabilization Protocol

The join or leave operations assume that routing informations are updated instantly. In practice, this assumption is unrealistic: routing information is distributed among all peers and communication latency is unavoidable. For that, some pointers and finger table entries may temporarily become inconsistent [2].

To preserve the correctness of the overlay despite these dynamic changes, the system periodically executes a **collection of maintenance procedures** collectively referred as *stabilization protocol*. Rather than immediately propagating topology updates to every node, the protocol progressively restores a consistent view of the identifier space through local interactions between neighboring peers. The stabilization protocol consists of four periodic procedures:

1. ***stabilize()*** - The stabilize procedure periodically check if the current successor remains the closest one. Each node contacts its successor and requests its predecessor. If this predecessor lies between the current node and its successor, it indicates that a new node has joined and it has an identifier belonging to such interval and should become the new successor. Otherwise, the current successor remains unchanged.

2. ***notify()*** - After executing the *stabilize()*, a node informs its successor of its existence by invoking a *notify()*. Upon receiving this notification, the successor updates its predecessor pointer whenever the notifying node is a closer predecessor than the currently stored one.

3. ***fix_fingers()*** - The finger table maintained by each node may become outdated after multiple join or leave operations. Rather than recomputing the entire routing table, the system periodically refreshes a single finger entry by performing a lookup for its corresponding identifier. Repeating this procedure over time gradually reconstructs the complete routing table.

4. ***check_predecessor()*** - The predecessor pointer is periodically verified by sending a heartbeat RPC request to the current predecessor. If no reply is received within a timeout interval, the predecessor is assumed to have failed and the pointer is reset to `null`. A subsequent execution of the stabilization procedure will automatically discover the correct predecessor.

The beauty thing of the stabilization protocol is that it is full decentralized. Every node executes the same procedures independently and periodically, without requiring any centralized coordinator or global knowledge of the overlay.

NOTE that the stabilization protocol does not guarantee that the overlay is always perfectly consistent. Instead, Chord guarantees *eventual consistency* on routing information: assuming that the rate of topology changes eventually decreases, repeated executions of the stabilization procedures progressively restore correct pointers and finger tables. Once convergence has been reached, lookup operations satisfy again the logarithmic complexity guarantees.

Example - Let's consider Figure 1. Let's put the focus on node 21 that has as successor node 32. A this time $successor(21) = 32$ while $predecessor(32) = 21$.

Assume now that a new node 27 joins the system. Since its identifier belongs to the interval

$$21 < 27 \leq 32$$

the new node must become responsible for the interval 21 (excluded) - 27 (included), while node 32 is responsible for 27 (excluded) - 32 (included). At this insertion time, only the so called *bootstrap node* 32 node is aware of its existence. Consequently, the routing state is temporarily inconsistent for node 21 since:

$$successor(21) = 32$$

although the correct successor should now be 27.

1) ***stabilize()*** - The first procedure invoked is *stabilize()*. As was previously said, the purpose of this procedure is to restores consistency. During its periodic execution, node 21 contacts its current successor 32 and requests its predecessor. Since 27 previously contacted 32 while joining the overlay, node 32 replies

$$predecessor(32) = 27$$

Node 21 then evaluates whether this predecessor lies in the interval

$$21 - 32$$

Since

$$21 < 27 < 32$$

the protocol concludes that 27 is closer than the current successor, and therefore updates its successor pointer:

$$successor(21) \leftarrow 27$$

2) ***nofify()*** - Then, node 21 invokes *notify()* on its new successor. This message informs 27 that 21 may be its predecessor. Upon receiving the notification, 27 compares the notifying node with its currently stored predecessor. Because 21 is the closest predecessor in the identifier space, it updates

$$predecessor(27) \leftarrow 21$$

During the next stabilization cycle, node 27 executes the same procedure by contacting its successor 32. Then it invokes *notify()* on 32, which consequently updates

$$predecessor(32) \leftarrow 27$$

At this point, the local neighborhood has converged to correct values:

$$21 \rightarrow 27 \rightarrow 32$$

3) ***fix_fingers()*** - The addition of 27 also affects the finger tables the other peers. For example, some finger entries of nodes 8, 14 and 21 that previously pointed directly, for instance, to 32 may now, for instance, have 27 as their correct destination. Rather than recomputing the entire table, each node periodically executes *fix_fingers()*, refreshing one entry at a time by performing a standard lookup. Consequently, routing information progressively converges while normal lookup operations continue to be served.

4) ***check_predecessor()*** - Now consider the situation in which node 27 crashes immediately after joining the overlay. Periodically, node 32 executes the function *check_predecessor()* by sending a heartbeat RPC to its predecessor to verify if it is alive or not. Failure is detected when no reply is received within the timeout interval.

At this point, it becomes easier to understand why Chord is said to provide *eventual consistency* of its routing tables. Immediately after node 27 joins the overlay, the routing information is temporarily inconsistent. For example, node 27 already knows that its successor is node 32, while node 21 still believes that node 32 is its immediate successor. Likewise, several finger tables across the network still contain outdated entries. As a result, different nodes temporarily have different views of the overlay, and lookup requests may follow longer-than-optimal paths. This situation is completely normal and is an expected consequence of Chord's decentralized design. There is no central coordinator responsible for instantly notifying every peer about topology changes. Instead, each node periodically executes the stabilization procedures, locally correcting its own routing information. As these local updates propagate through the network, the outdated information gradually disappears. **Eventually, provided that no additional joins, departures or failures occur for some time, every node converges to the same view of the identifier space**. At that point, the routing information is once again consistent and lookup operations recover their expected $O(\log N)$ complexity.

3.7 Requirement Analysis

The design choices introduced by Chord is demonstrated to satisfy the principal requirements of large-scale distributed systems discussed in section 1.2.3. The principal theoretical properties of Chord are summarized below [1] [2].

Requirement	**Chord Evaluation**
Efficient Resource Discovery	Chord chases this requirement through its finger tables. Lookup requests are forwarded using distributed routing entries, allowing any resource to be located in an expected $O(\log N)$ routing hops instead of the linear search required by unstructured systems.
Scalability	Each participating peer maintains only $O(\log N)$ routing entries, independently of the total network size. Consequently, both memory consumption and routing overhead grow logarithmically as new peers join the overlay, allowing Chord to efficiently scale to even in large-scale situations.
Dynamic Membership	Chord naturally supports node joins, graceful departures and unexpected failures. Thanks to consistent hashing, only the resources belonging to the responsibility interval of the affected node are relocated, requiring the migration of an expected $O(K/N)$ keys. Routing information is progressively restored through the stabilization protocol.
Load Balancing	Consistent hashing distributes node identifiers and resource identifiers approximately uniformly over the identifier space. Assuming a uniform hash function, every peer stores approximately K/N resources with high probability, preventing load concentration while improving the utilization of the available storage capacity.
Fault Tolerance	Chord tolerates crash failures through periodic maintenance procedures. Neighboring peers detect failures through heathbeat, timeout-based RPCs, while the stabilization protocol progressively reconstructs successor pointers, predecessor pointers and finger tables. Although routing information may temporarily become inconsistent, repeated executions of the maintenance procedures eventually restore a correct overlay.

3.8 Security Implications

Chord do not make a fail-arbitrary failure over the system: it is assumed that peers honestly execute the protocol while maybe due to volunteer leaving, to internal crashing or to network issues, exits. But no assumption over byzantine nodes is made: system assumes that active peer behaves correctly maintain their routing information and honestly forward lookup requests and report successor and predecessor relationships. However, since P2P systems are, in many cases, open environments tha follows that any participant may join the overlay [1] [2].

Several attacks have therefore been proposed in the literature against DHT. They are briefly described them in the following.

Sybil Attack - One of the most known attacks against DHT is the *sybil attack*.

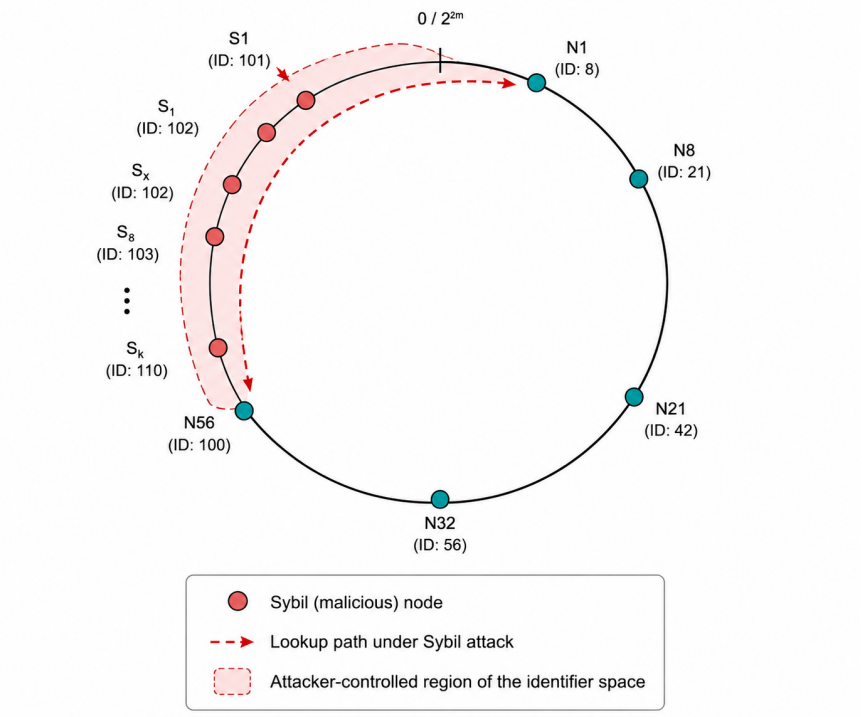

Figure 2: Chord ring - Sybil attack

The attack consists creating a large number of byzantine peers. Since Chord assigns node identifiers through hashing, the protocol assumes that every identifier corresponds to a different physical participant. An attacker capable of forging many identities can violate this assumption by occupying multiple positions within the identifier space by pre-calculating ad-hoc forged indexes designed to

occupy specific positions in the ring. This implies that, since the fake identities are strategically positioned along the identifier ring, the attacker can substantially increase its influence over routing decisions [6].

The consequences are significant: a malicious participant controlling many identifiers may become responsible for a large segment of the identifier space, permitting to him to intercept lookup requests, manipulate routing information or selectively refuse to forward messages.

Eclipse Attack - An Eclipse attack aims to isolate one honest nodes from the rest of the overlay by surrounding it with byzantine neighbors. Here, its goal is not to cover a portion of the ring as in the sybil strategy, but to isolate specific nodes by surrounding an honest node with byzantine nodes [7].

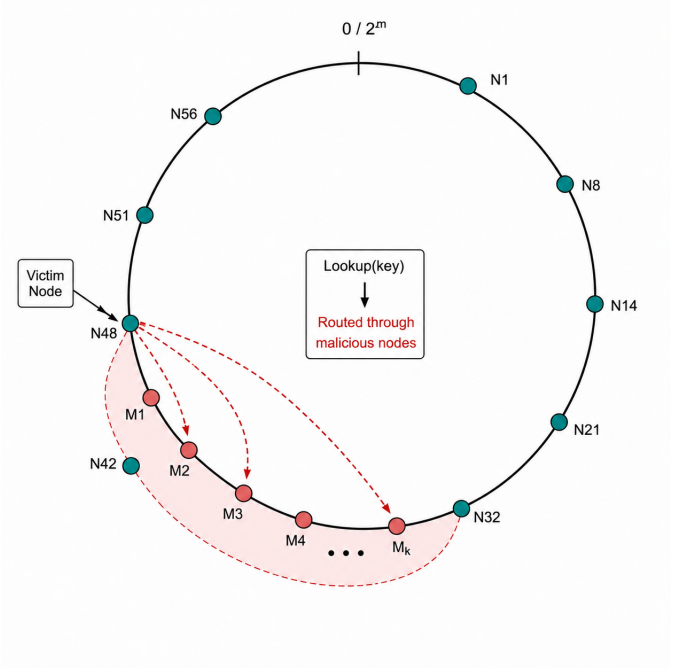

Figure 3: Chord ring - Sybil attack

Since every node maintains routing information concerning its successor, predecessor and finger table entries, if an attacker succeeds in replacing a sufficiently large number of these routing entries with malicious nodes, every lookup initiated by the victim is forced to traverse attacker-controlled peers. As a consequence, malicious nodes may observe delay, modify or discard lookup requests. In the worst case, the victim becomes logically disconnected from the remaining overlay despite maintaining apparently valid routing information.

Routing Attack - Routing attacks target the lookup mechanism used to locate resources in the overlay. An attacker interferes with the forwarding of lookup requests by causing them to follow incorrect paths. The routing process can be attaccked in several ways, the main are briefly reported below [7].

- **Lookup misdirection** - A malicious peer deliberately forwards lookup requests toward incorrect destinations, increasing lookup latency or preventing resources from being located.
- **Message dropping** - Rather than forwarding a received lookup request, the byzantine peer may simply discard it. Since the requesting peer generally expects intermediate peers to cooperate, the lookup eventually goes in timeout considered unsuccessful. Repeated message dropping can effectively make an entire portion of the identifier space unreachable, producing, by all the effect, a denial-of-service for all resources managed by that segment.
- **Routing table poisoning** - Malicious nodes advertise incorrect successor, predecessor or finger table information, causing honest peers to construct wrong (inconsistent) routing tables. Honest nodes accepting these updates gradually construct inconsistent routing tables, causing future lookups to traverse malicious nodes more frequently.
- **Selective forwarding** - Arbitrarial. Rather than dropping every request, an attacker forwards only selected messages while discarding others, making the attack considerably more difficult to detect.

Countermeasures - Although Chord was not specifically designed to handle byzantine participants, some mitigations have been proposed by the literature to improve its security for such class of problems.

One of the most effective mitigations is **access control**. In the protocol, node identifiers are generated through hashing, implicitly assuming that each identifier corresponds to a distinct physical participant. An attacker capable of generating an arbitrary number of identities can therefore obtain arbitrary control over the identifier space. Mitigation consist to require joining node to be certified by a Certified Authority (CA) or generated through computationally expensive mechanisms, as Proof-of-Work (PoW) with the objective to increases the cost and/or effort of creating multiple identities, reducing feasibility.

Another mechanism that reduce the attack surface is **replication**. Instead of storing each resource exclusively on its immediate successor, multiple replicas are maintained on a set of consecutive successor nodes. As a consequence, the failure or compromise of a single peer does not necessarily result in data loss or resource unavailability. In other words, a single resource is replicated in many peer. Replication therefore improves both fault tolerance and resilience against malicious nodes attempting to target resources. The main drawback is

the additional storage and maintenance overhead, as multiple peers must store and periodically synchronize the same data, reducing the overall efficiency of the system.

Another layer that can be added is about **securing stabilization**. Chord periodically updates successor and predecessor pointers and finger table entries through stabilization protocol. If routing updates are accepted without verification, malicious peers may advertise incorrect routing information and gradually poison the overlay topology. Authenticating routing messages and periodically validating routing entries against multiple peers help detect inconsistent information before it propagates throughout the overlay.

Last, but not least, notable mitigation is about incorporating to the DHT implementations a **reputation systems** or trust mechanisms that continuously evaluate the behavior of neighboring peers. Nodes that repeatedly fail to forward requests, inconsistent routing information or otherwise deviate from the protocol receive lower trust scores and are progressively excluded from routing decisions. Although reputation systems cannot completely eliminate malicious participants, they reduce their long-term influence by encouraging routing through peers that have consistently demonstrated correct behavior.

Despite these improvements, providing strong security guarantees in fully decentralized P2P system remains an open research problem.

4 Applications

Since its introduction in 2001, Chord has been one of the most influential DHT protocols in structured P2P systems. Although relatively few production systems directly implement it, its underlying design principles have, directly and trasversely, influenced the evolution of large-scale systems.

In this last section are presented several representative examples of Chord's practical application.

CoralCDN - A peer-assisted *Content Distribution Network* (CDN) introduced in 2004 to improve the scalability and availability of web content through a decentralized replication infrastructure. Unlike traditional CDNs, which rely on central solutions, CoralCDN distributes content in a decentralized way among a large number of participating peers, allowing replicated objects to be served directly by the network while reducing the load on origin servers without either a coordinator nor a global knowledge of the network [8].

The system employs a Chord-like DHT to efficiently locate cached content. Each web object is assigned an identifier obtained by hashing its URL. The DHT deterministically links object identifiers to responsible peers. Rather than storing the web objects directly in the DHT, CoralCDN stores only directory information indicating which peers currently stores a given object. Once the appropriate peer has been lookuped through the overlay, the requested content is transferred directly from one of the available replicas.

This implementation extends the original Chord design through the introduction of *Distributed Sloppy Hash Tables* (DSHTs), which relax the strict identifier ownership imposed by traditional DHTs. Instead of always contacting the unique node responsible for an identifier, lookups may be satisfied by nearby peers belonging to the same locality cluster. Exploiting network locality reduces latency, decreases inter-domain traffic and improves overall system performance while preserving the scalability of the structured overlay.

OpenDHT - OpenDHT is an open-source Distributed Hash Table (DHT) implementaion. Developed as a middleware platform, it offers a practical implementation of structured P2P networks, enabling applications to store, retrieve and discover information without relying on centralized components [9].

Like Chord, organizes peers into a structured overlay network where both nodes and data are assigned identifiers through cryptographic hash functions. The platform supports a wide range of distributed applications, including decentralized storage, service discovery and messaging systems deployments.

Influence on Modern Distributed Storage Systems - Although relatively few production systems directly implement Chord, it has profoundly influenced the architecture of modern distributed storage systems. In particular, the concepts of consistent hashing, ring-based data partitioning, and decentralized resource placement have proven illuminating, becoming fundamental building blocks of contemporary distributed databases.

One of the main examples is ***Amazon Dynamo***, a highly available key-value store developed to support Amazon's large-scale services. Dynamo organizes data using a logical identifier ring based on consistent hashing [10].

The same architectural principles were later adopted by ***Apache Cassandra***, an open-source distributed database originally inspired by Amazon Dynamo. Cassandra employs consistent hashing to partition data across a logical ring and replicates each data partition across multiple nodes. Similar to Dynamo, routing is based on decentralized membership services rather than Chord's logarithmic lookup algorithm, allowing any node to coordinate client requests [11].

Another representative example is ***Riak***, a distributed NoSQL database designed for scalable and fault-tolerant storage. Riak also partitions data through consistent hashing and distributes replicas across multiple physical nodes. Automatic rebalancing during node joins and leaves enables the system to maintain a balanced distribution of data while preserving service availability [12].

Although these systems no longer implement Chord's routing protocol, they preserve several of its fundamental concepts.

BitTorrent and the Adoption of Kademlia - BitTorrent is one of the most widely used file-sharing systems. Early versions relied on centralized trackers to maintain information about peers participating in a torrent, creating a potential single point of failure. To eliminate this dependency, BitTorrent introduced the *Mainline Distributed Hash Table*, which provides fully decentralized peer discovery. Rather than adopting Chord, the Mainline DHT is based on the *Kademlia* protocol, a DHT designed for file-sharing P2P environments [13].

Chord and Kademlia has some touchpoints as: hash functions to assign identifiers, deterministic resource placement and logarithmic lookup complexity. However, they uses two different routing strategies. Kademlia measures the distance between identifiers using the bitwise xor and maintains routing information in structures known as *k-buckets*. Each k-bucket stores multiple peers within a given distance range, providing many alternative paths toward a destination.

Furthermore, Kademlia updates routing information opportunistically during normal protocol operations, resulting in lower maintenance overhead than the periodic stabilization procedures required by Chord.

5 Conclusion

Chord introduced an elegant and scalable solution to one of the fundamental problems in distributed systems: locating resources in a decentralized environment. By organizing participating peers into a structured overlay network and employing consistent hashing, Chord provides deterministic resource placement while requiring each node to maintain only a logarithmic amount of routing information. As a result, resource discovery can be performed efficiently without centralized coordination, making the protocol a landmark contribution to the design of Distributed Hash Tables.

The simplicity of Chord's architecture, together with its provable efficiency and scalability, established many of the principles that continue to guide the development of P2P systems.

Although relatively few modern systems directly implement the original Chord protocol, its influence extends far beyond its initial design. More than two decades after its introduction, Chord remains one of the most influential Distributed Hash Table protocols ever proposed. Its architectural principles have shaped the evolution of distributed storage systems, decentralized applications and peer-to-peer networks, making it both a foundational research contribution and an enduring reference for the design of scalable distributed systems.large-scale decentralized architectures.

References

[1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. Maarten van Steen, 2023.

[2] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup protocol for internet applications," *IEEE/ACM Transactions on Networking*, vol. 11, no. 1, pp. 17–32, Feb. 2003.

[3] National Institute of Standards and Technology, "Secure hash standard (shs)," FIPS PUB 180-4, 2015.

[4] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, "A scalable content-addressable network," *ACM SIGCOMM Computer Communication Review*, vol. 31, no. 4, pp. 161–172, 2001.

[5] D. Karger, E. Lehman, T. Leighton, M. Levine, D. Lewin, and R. Panigrahy, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web," in *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC)*, 1997, pp. 654–663.

[6] J. R. Douceur, "The sybil attack," in *Proceedings of the First International Workshop on Peer-to-Peer Systems (IPTPS)*. Springer, 2002, pp. 251–260.

[7] A. Singh, M. Castro, P. Druschel, and A. Rowstron, "Defending against eclipse attacks on overlay networks," in *Proceedings of the ACM SIGOPS European Workshop*, 2004.

[8] M. J. Freedman, "Experiences with coralcdn: A practical peer-to-peer content distribution network," Ph.D. dissertation, New York University, 2005.

[9] S. Riffon *et al.*, "Opendht: A public distributed hash table service," in *IEEE International Conference on Peer-to-Peer Computing (P2P)*, 2016.

[10] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007, pp. 205–220.

[11] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35–40, 2010.

[12] Basho Technologies, "Riak kv documentation," https://docs.riak.com/riak/kv/, 2015.

[13] BitTorrent.org, "Bep 5: Dht protocol," https://www.bittorrent.org/beps/bep$_0$005.*html*, 2008.